instance, the SHARC family can represent numbers in 32-bit fixed point, a mode that is common in digital audio applications. This makes the 2^{32} quantization levels spaced uniformly over a relatively small range, say, between -1 and 1. In comparison, floating point notation places the 2^{32} quantization levels logarithmically over a huge range, typically $\pm 3.4 \times 10^{38}$. This gives 32-bit fixed point better *precision*, that is, the quantization error on any one sample will be lower. However, 32-bit floating point has a higher *dynamic range*, meaning there is a greater difference between the largest number and the smallest number that can be represented.

C versus Assembly

DSPs are programmed in the same languages as other scientific and engineering applications, usually *assembly* or *C*. Programs written in assembly can execute faster, while programs written in C are easier to develop and maintain. In traditional applications, such as programs run on personal computers and mainframes, C is almost always the first choice. If assembly is used at all, it is restricted to short subroutines that must run with the utmost speed. This is shown graphically in Fig. 28-9a; for every traditional programmer that works in assembly, there are approximately *ten* that use C.

However, DSP programs are different from traditional software tasks in two important respects. First, the programs are usually much shorter, say, one-hundred lines versus ten-thousand lines. Second, the execution speed is often a critical part of the application. After all, that's why someone uses a DSP in the first place, for its blinding speed. These two factors motivate many software engineers to switch from C to assembly for programming Digital Signal Processors. This is illustrated in (b); nearly as many DSP programmers use assembly as use C.

Figure (c) takes this further by looking at the revenue produced by DSP products. For every dollar made with a DSP programmed in C, two dollars are made with a DSP programmed in assembly. The reason for this is simple; money is made by outperforming the competition. From a pure performance standpoint, such as execution speed and manufacturing cost, assembly almost always has the advantage over C. For instance, C code usually requires a larger memory than assembly, resulting in more expensive hardware. However, the DSP market is continually changing. As the market grows, manufacturers will respond by designing DSPs that are *optimized* for programming in C. For instance, C is much more efficient when there is a large, general purpose register set and a unified memory space. These future improvements will minimize the difference in execution time between C and assembly, and allow C to be used in more applications.

To better understand this decision between C and assembly, let's look at a typical DSP task programmed in each language. The example we will use is the calculation of the *dot product* of the two arrays, $x[\,]$ and $y[\,]$. This is a simple mathematical operation, we multiply each coefficient in one